AI ASSISTED CODING

ASSIGNMENT-12.5

NAME:M.AKHIL REDDY

ROLL NO:2303A52315

BATCH-45

Task Description #1 (Sorting – Merge Sort Implementation)

• Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

• Instructions:

o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.

o Ask AI to include time complexity and space complexity in the function docstring.

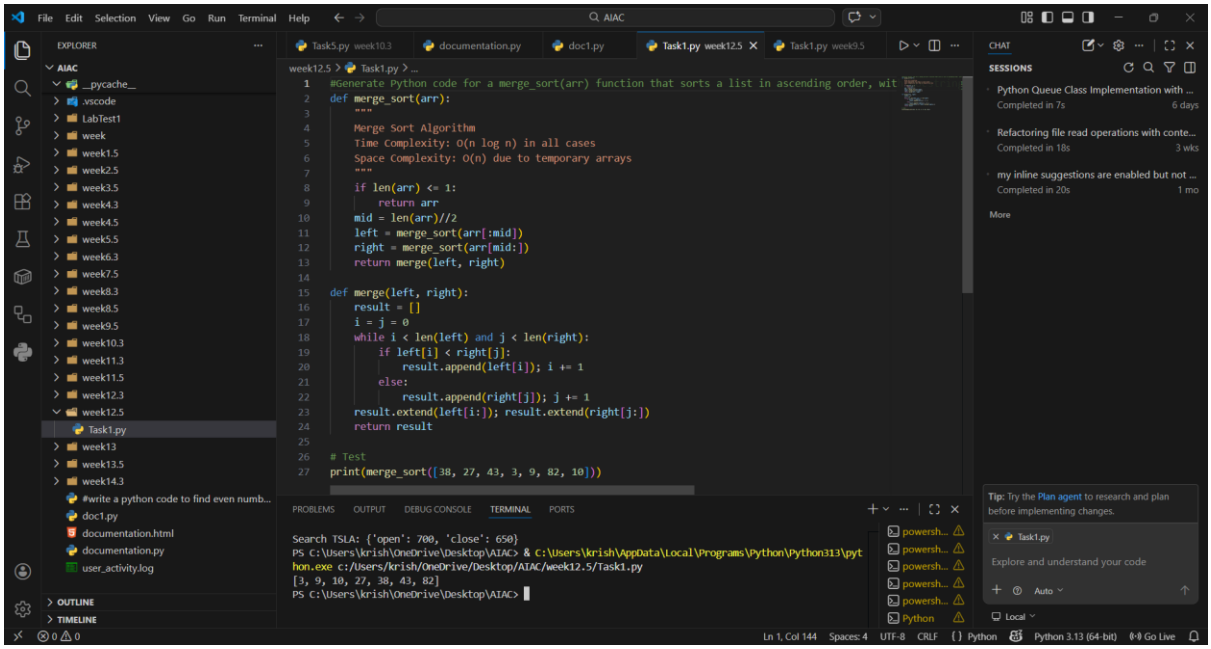
o Verify the generated code with test cases.

• Expected Output:

o A functional Python script implementing Merge Sort with proper documentation.

PROMPT: "Generate Python code for a merge_sort(arr) function that sorts a list in ascending order, with docstrings including time and space complexity."

CODE AND OUTPUT:



EXPLANATION:

Merge Sort is a divide-and-conquer algorithm that splits the list into halves, sorts them recursively, and merges results. It guarantees **O(n log n)** performance regardless of input order. The extra space used for temporary arrays makes its space complexity **O(n)**. It is stable and efficient for large datasets

Task Description #2 (Searching – Binary Search with AI Optimization)

- **Task: Use AI to create a binary search function that finds a target element in a sorted list.**
- **Instructions:**

o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.

o Include docstrings explaining best, average, and worst-case complexities.

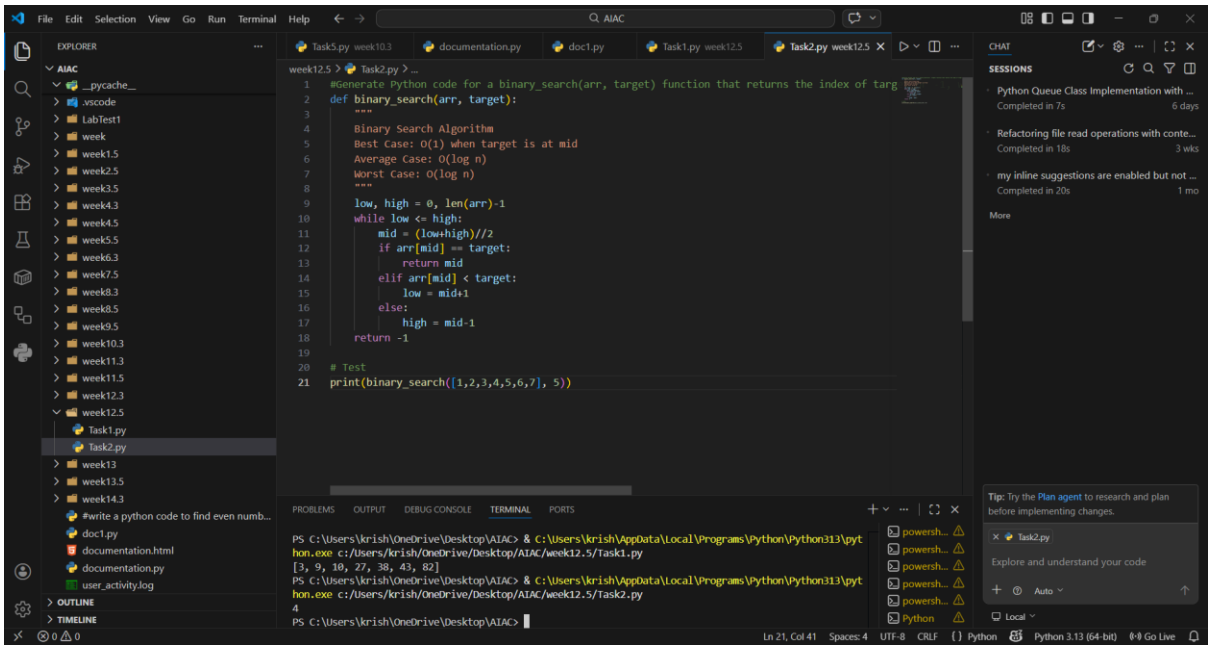
o Test with various inputs.

• **Expected Output:**

o Python code implementing binary search with AI-generated comments and docstrings.

PROMPT: " Generate Python code for a binary_search(arr, target) function that returns the index of target or -1, with docstrings explaining best, average, and worst-case complexities."

CODE AND OUTPUT:



EXPLANATION:

Binary Search works on sorted lists by repeatedly halving the search space. It is highly efficient with **O(log n)** complexity in average and worst cases. The best case occurs when the target is found at the midpoint immediately. It is ideal for large datasets where quick lookups are required.

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.

2. Sort appointments based on time or consultation fee.

Student Task

• Use AI to recommend suitable searching and sorting

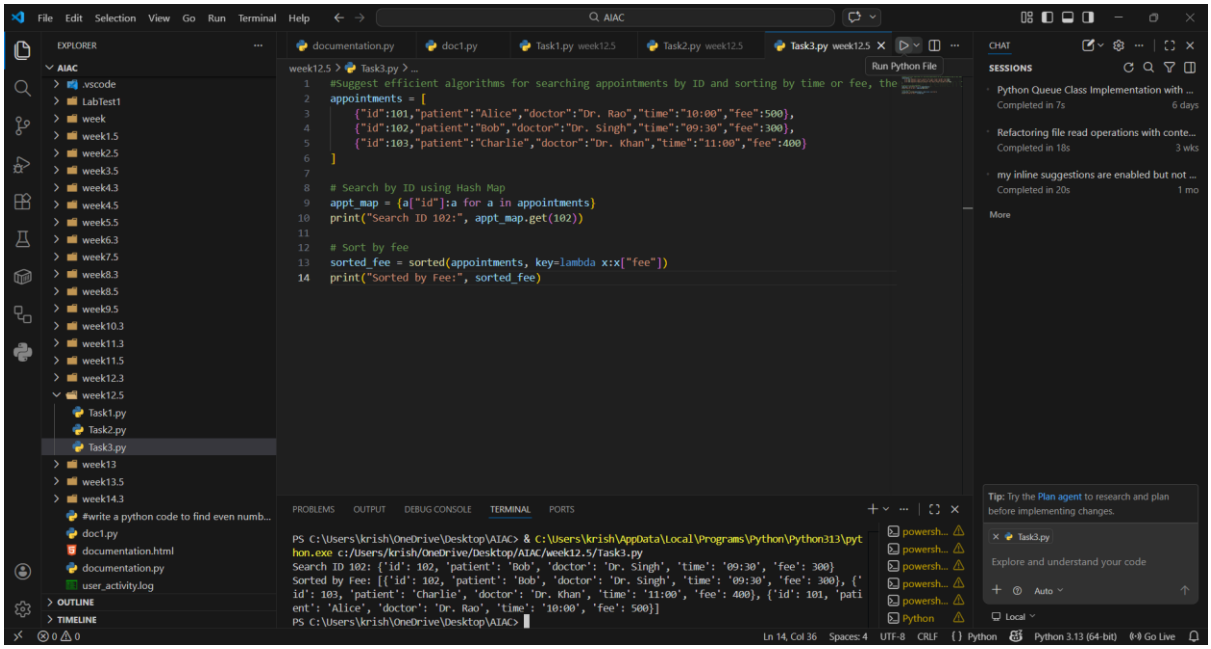
algorithms.

• Justify the selected algorithms.

• Implement the algorithms in Python.

PROMPT: "Suggest efficient algorithms for searching appointments by ID and sorting by time or fee, then implement them in Python.""

CODE AND OUTPUT:



EXPLANATION:

Searching by appointment ID is best handled with a **Hash Map** for O(1) lookups. Sorting by time or fee can be efficiently done using **Merge Sort** or Python's built-in sorted() (Timsort). This ensures quick retrieval and stable ordering, which is crucial for healthcare scheduling.

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket

ID, passenger name, train number, seat number, and travel date. The

system must:

1. Search tickets using ticket ID.

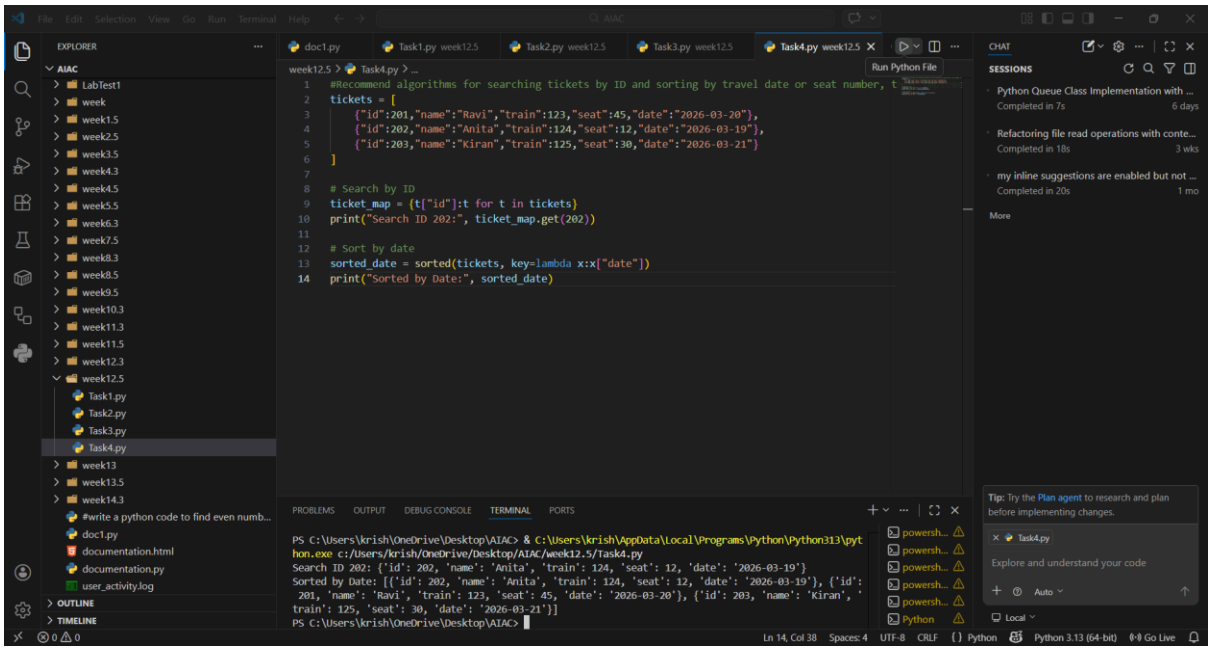
2. Sort bookings based on travel date or seat number.

Student Task

- **Identify efficient algorithms using AI assistance.**
- **Justify the algorithm choices.**
- **Implement searching and sorting in Python.**

PROMPT: "Recommend algorithms for searching tickets by ID and sorting by travel date or seat number, then implement them in Python."

CODE AND OUTPUT:



EXPLANATION:

Ticket searches by ID are optimized with **Hash Maps** for instant retrieval. Sorting by travel date or seat number can be done with **Merge Sort** or built-in sorting for efficiency. This ensures passengers' bookings are managed quickly and accurately

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The

system needs to:

1. Search allocation details using student ID.

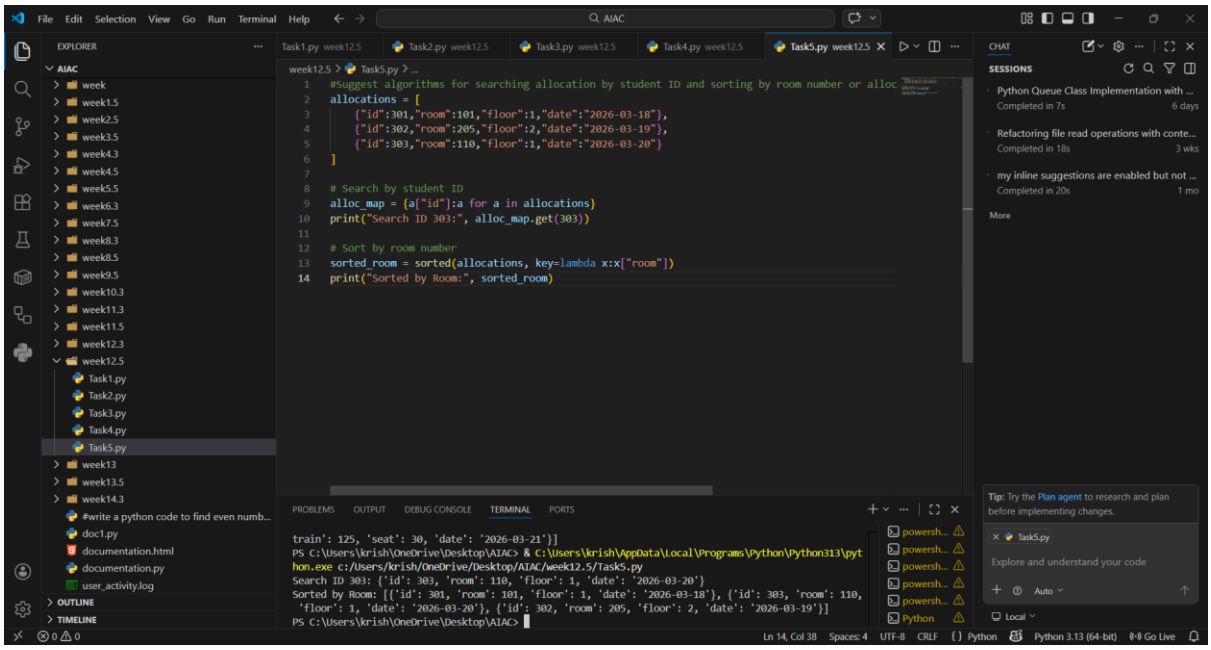
2. Sort records based on room number or allocation date.

Student Task

- **Use AI to suggest optimized algorithms.**
- **Justify the selections.**
- **Implement the solution in Python.**

PROMPT: "Suggest algorithms for searching allocation by student ID and sorting by room number or allocation date, then implement them in Python."

CODE AND OUTPUT:



EXPLANATION:

Student allocations are best searched using **Hash Maps** for O(1) access. Sorting by room number or allocation date can be done with **Merge Sort** or built-in sorting. This ensures hostel management can quickly assign and track rooms.

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title,

genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.

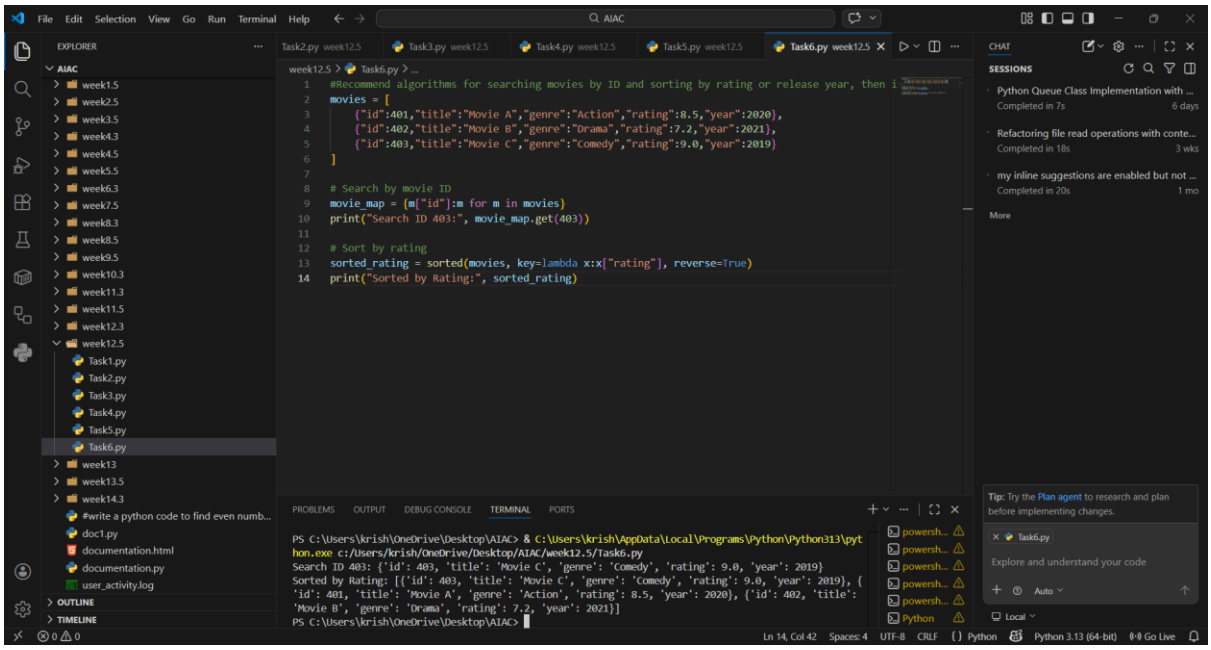
2. Sort movies based on rating or release year.

Student Task

- **Recommend searching and sorting algorithms using AI.**
- **Justify the chosen algorithms.**
- **Implement Python functions.**

PROMPT: "Recommend algorithms for searching movies by ID and sorting by rating or release year, then implement them in Python."

CODE AND OUTPUT:



EXPLANATION:

Movie searches by ID are efficiently handled with **Hash Maps**. Sorting by rating or release year can be done with **Quick Sort** or built-in sorting. This ensures users can quickly find top-rated or latest movies

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers

need to:

1. Search crop details using crop ID.

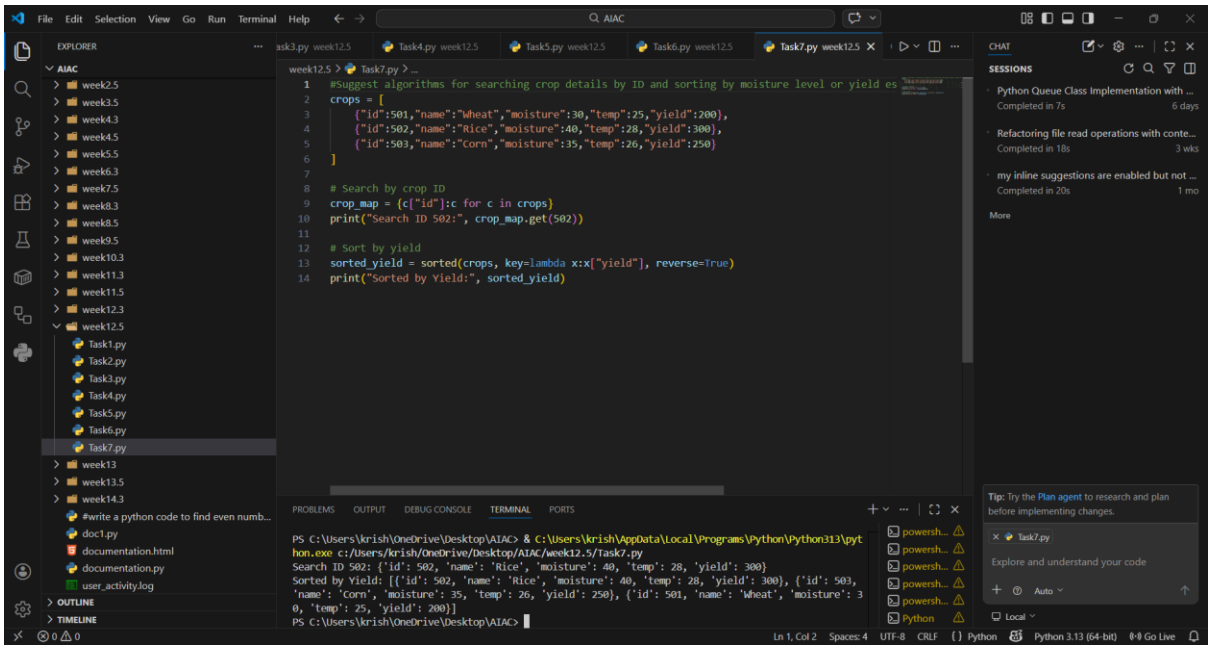
2. Sort crops based on moisture level or yield estimate.

Student Task

- **Use AI-assisted reasoning to select algorithms.**
- **Justify algorithm suitability.**
- **Implement searching and sorting in Python.**

PROMPT: "Suggest algorithms for searching crop details by ID and sorting by moisture level or yield estimate, then implement them in Python. "

CODE AND OUTPUT:



EXPLANATION:

Crop details are best searched using **Hash Maps** for instant retrieval. Sorting by moisture or yield can be done with **Merge Sort** or built-in sorting. This helps farmers prioritize crops based on productivity and soil conditions.

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID,

airline name, departure time, arrival time, and status. The system

must:

1. Search flight details using flight ID.

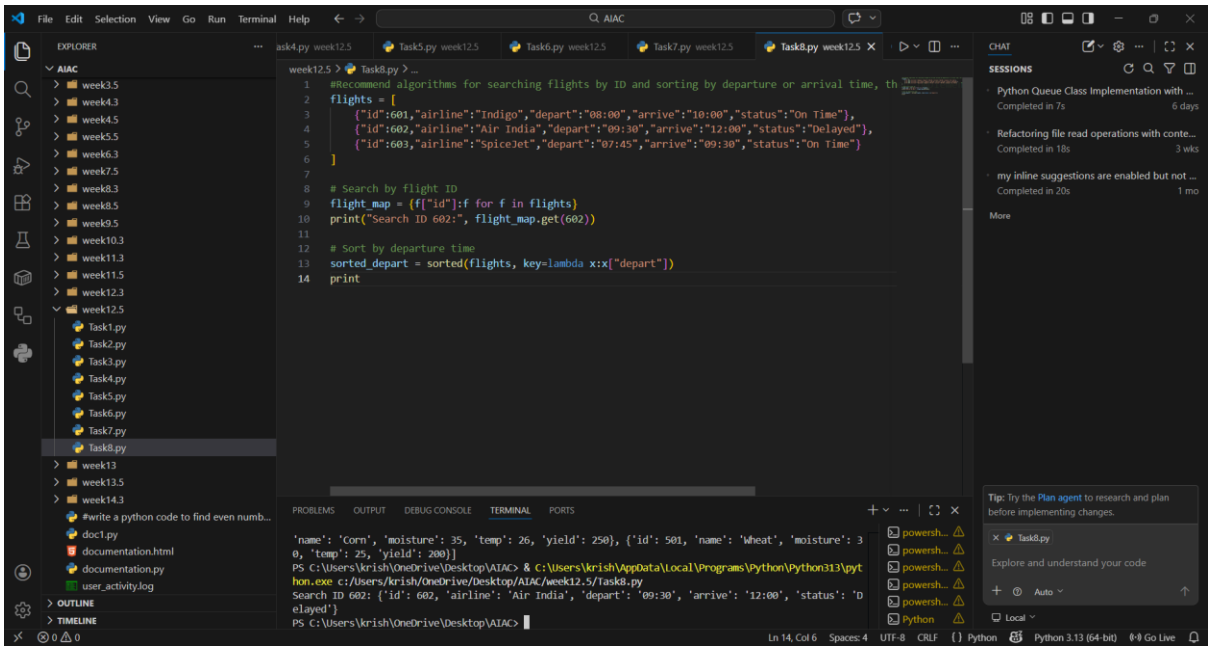
2. Sort flights based on departure time or arrival time.

Student Task

- **Use AI to recommend algorithms.**
- **Justify the algorithm selection.**
- **Implement searching and sorting logic in Python.**

PROMPT: "Recommend algorithms for searching flights by ID and sorting by departure or arrival time, then implement them in Python."

CODE AND OUTPUT:



EXPLANATION:

Searching flight details by **flight ID** is best handled using a **Hash Map**, since it allows constant-time lookups regardless of dataset size. Sorting flights by **departure or arrival time** can be efficiently done using **Merge Sort** or Python's built-in sorted() function, which is based on Timsort. These algorithms ensure stable and accurate ordering, which is critical for managing flight schedules. The combination of hash-based search and efficient sorting provides both speed and reliability in real-time airport operations.